High level Design

on

IaC Provisioning for Non-Profit System

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Neha Tamboli	EN22CS301641
2	Riddhima Taose	EN22CS303042
3	Uday Patidar	EN23CS3L1023
4	Rohan Rawat	EN23CS3L1016
5	Naman Dwivedi	EN22CS301627

Group Name: 04D6

Project Number: DO-15

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience.
   1.3. System overview.

2. **System Design.**
   2.1. Application Design.
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design.
   2.5. Key Design Considerations.
   2.6. API Catalogue.
3. **Data Design.**
   3.1 Data Flow Description.
   3.2 Data Access Mechanism.
   3.3 Data Retention Policies.
   3.4 Data Migration.
4. **Interfaces.**
5. **State and Session Management.**
6. **Caching.**
7. **Functional Requirements.**
   7.1 State References User Authentication & Authorization.
   7.2 Application Configuration Management.
   7.3 Processing & Monitoring.
   7.4 System Operations.
   7.5 Docker.
   7.6 CI/CD Pipeline.
   7.7 CI/CD Strategy.
8. **Non-Functional Requirements.**
   8.1 Security Aspects.
   8.2 Performance Aspects.
9. **References.**
10. **GitHub.**

1. Introduction

1.1. Scope of the document

This document defines the high-level architecture and design for an AI-powered Non-Profit Support Triage system. It focuses on automated processing of communication messages using AI agent workflows for classification, entity extraction, and response generation, without persistent database storage, utilizing in-memory vector processing.

1.2. Intended Audience

- Project Supervisors: To evaluate technical feasibility and design accuracy
- AI/ML Engineers: For understanding agent orchestration and LangChain integration
- DevOps Engineers: For Docker/Kubernetes deployment workflows
- System Architects: To review Streamlit dashboard and ChromaDB architecture

1.3. System overview

The system automates triage of non-profit organization support messages (donation requests, volunteer inquiries, complaints) through an intelligent pipeline of AI agents. Incoming messages are classified by intent and urgency, key entities (Indian names, ₹ amounts, locations, dates) are extracted, and contextual responses are generated. The Streamlit dashboard provides real-time analysis visualization, with ChromaDB handling in-memory vector storage. Containerized deployment via Docker and Kubernetes ensures scalability and consistency across environments.

2. System Design

2.1. Application Design

The application follows an agent-based microservices architecture where specialized AI agents handle distinct processing tasks. The Classification Agent identifies message intent (donation, volunteer, complaint, general inquiry) and urgency level (high/medium/low). The Entity Extraction Agent identifies structured information including Indian names, ₹ amounts, locations, and dates. The Response Generation Agent creates contextual replies based on classification and extracted entities. These agents are orchestrated by LangChain framework and presented through a Streamlit web interface, all containerized within Docker for consistent deployment.

2.2 Process Flow

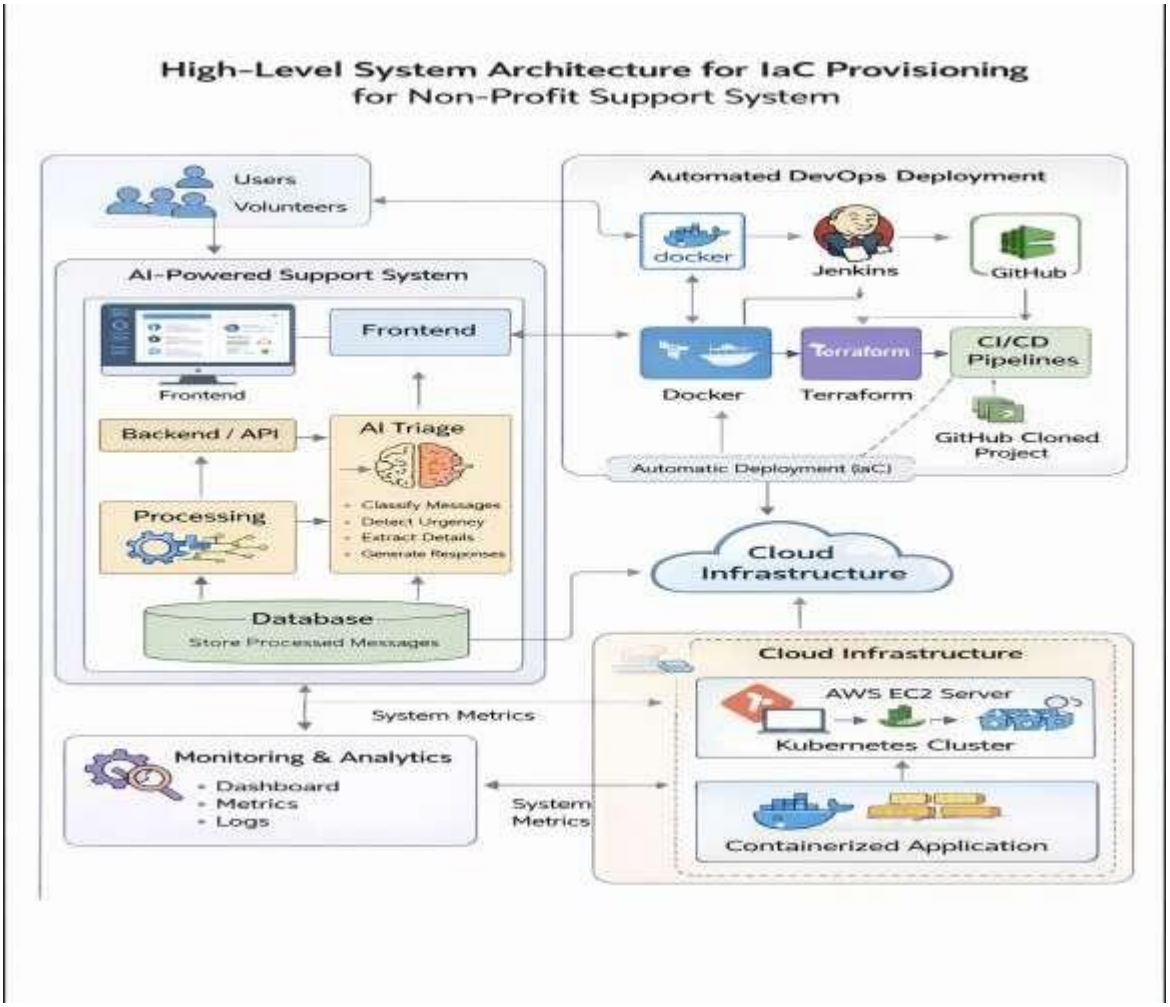
The process begins at the developer's machine, where the application code and Infrastructure as Code (IaC) scripts are developed and maintained. The code includes the AI-powered Non-Profit Support Triage system along with Docker configuration files for containerization.

Once development is complete, the code is pushed to a GitHub repository, which acts as the central version control system. This action triggers the CI/CD pipeline using tools such as GitHub Actions or Jenkins. The pipeline automatically performs tasks such as code validation, dependency installation, and Docker image creation.

After successful build and testing, the Docker image is deployed either on a local Docker environment or on cloud infrastructure such as AWS EC2. Terraform scripts can be used to provision the required infrastructure resources dynamically.

The application is then executed inside Docker containers, making it accessible through a web interface. The system processes incoming messages, performs classification and analysis, and generates responses. All processed data is stored and monitored through the system dashboard and analytics module.

This automated workflow ensures a consistent, scalable, and reliable deployment process while reducing manual intervention.



2.3 Information Flow

Information flows from the GitHub repository, where the code and IaC scripts are stored, to the CI/CD pipeline, which builds Docker images and provisions infrastructure using Terraform. The application is then deployed on Docker or cloud environments like AWS EC2. User messages are processed by the system, analyzed, and stored in the database, and the results are displayed on the dashboard. Terraform maintains a state file to track and manage the deployed resources.

2.4 Components Design

- **GitHub Repository:** Acts as the central version control system where application code, Docker configuration, and Infrastructure as Code (IaC) scripts are stored. It serves as the single source of truth and enables collaboration among team members.
- **CI/CD Pipeline (GitHub Actions/Jenkins):** Functions as the automation engine that triggers build and deployment processes whenever code is pushed to the repository. It handles tasks such as code validation, Docker image creation, and deployment automation.
- **Terraform:** Used as the core Infrastructure as Code tool to provision and manage cloud resources such as AWS EC2 instances. It ensures that infrastructure setup is automated, consistent, and repeatable.
- **Docker:** Used for containerizing the application, including the frontend and backend components. It ensures portability and consistent execution across different environments.
- **AWS EC2 / Cloud Environment:** Provides the hosting infrastructure where Docker containers are deployed and executed, making the application accessible via a public IP.
- **Application (Non-Profit Support Triage System):** The main system that processes user messages, performs classification, extracts information, and generates responses through an interactive dashboard.

2.5 Key Design Considerations

- **Modularity:** The system is designed using a modular approach where different components such as frontend, backend processing, AI logic, and infrastructure scripts are separated. This improves maintainability and allows easy updates or enhancements.
- **Scalability:** The application is containerized using Docker and can be deployed on cloud platforms like AWS, allowing the system to handle increasing user requests efficiently by scaling resources when needed.
- **Consistency:** By using Docker and Infrastructure as Code (Terraform), the system ensures that development, testing, and production environments remain identical, eliminating configuration mismatches.
- **Automation:** CI/CD pipelines automate the build and deployment process, reducing manual intervention and minimizing human errors.

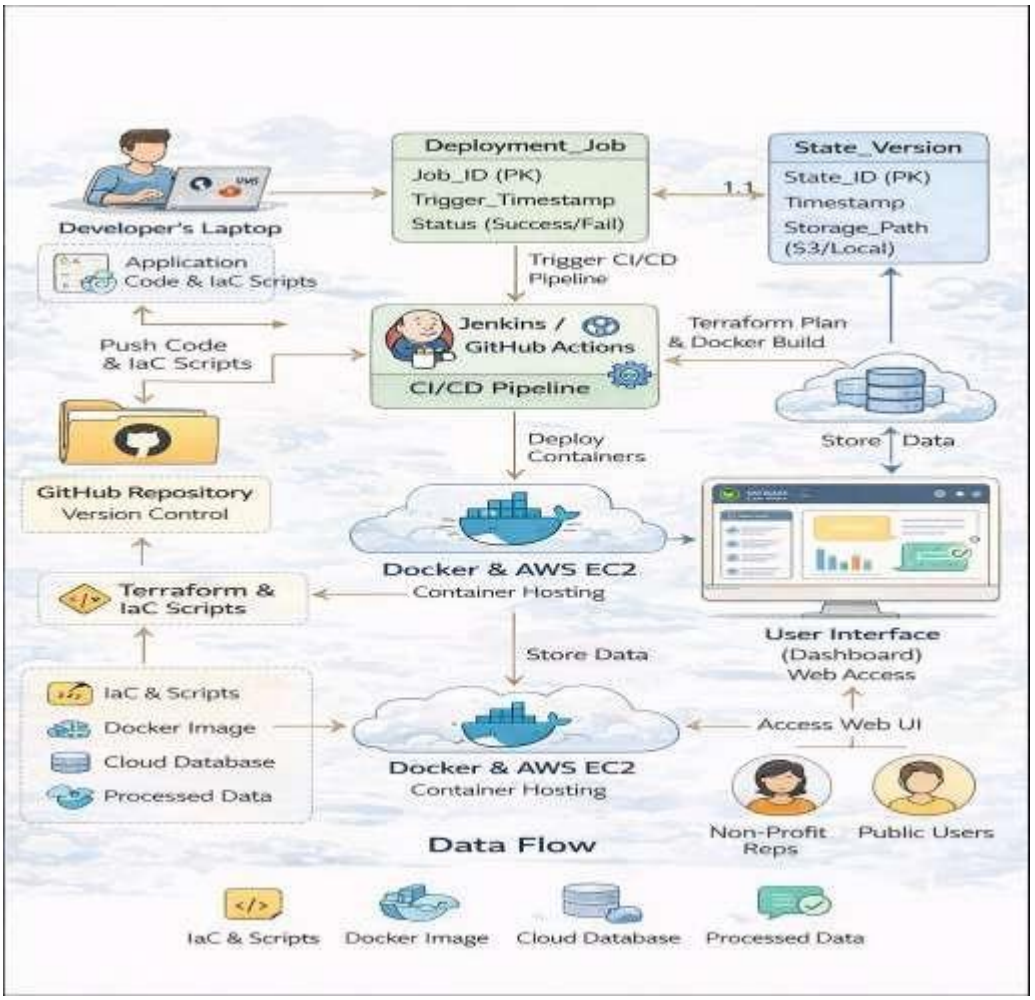
- **Security:** Sensitive data such as API keys and credentials are managed using environment variables, ensuring secure handling of information.

2.6 API Catalogue

- **LLM API (Language Model API):** Used for processing user messages, including classification, entity extraction, and response generation in the Non-Profit Support Triage System.
- **Docker API:** Used to manage container lifecycle operations such as building images, running containers, and monitoring application services.
- **Cloud API (AWS EC2 API):** Used to provision and manage cloud infrastructure resources where the application is deployed.
- **CI/CD Pipeline API (GitHub Actions/Jenkins):** Used to automate build, test, and deployment processes whenever code changes are pushed to the repository.

3. Data Design

3.1 Data Flow Description



Data Flow

- The data flow of the system begins from the developer's machine, where application code and Infrastructure as Code (IaC) scripts are created. These scripts, along with the application code, are pushed to the GitHub repository, which acts as the central source of truth.
- Once the code is pushed, the CI/CD pipeline (GitHub Actions or Jenkins) is triggered automatically. The pipeline processes the code by building Docker images and executing Terraform scripts for provisioning infrastructure.
- The Docker containers are then deployed on cloud infrastructure such as AWS EC2. The application becomes accessible through a web-based user interface.
- Users interact with the system by submitting messages through the frontend. These messages are sent to the backend, where they are processed using the AI-based triage system. The system performs classification, urgency detection, and information extraction.
- The processed data is then stored in the database and displayed on the dashboard for monitoring and analysis. Additionally, Terraform maintains a state file to track infrastructure changes and ensure consistency between the code and deployed resources.

3.2 Data Access Mechanism

Access to code and infrastructure is managed through GitHub using role-based permissions. The CI/CD pipeline securely accesses the repository using tokens to build and deploy the application. Terraform uses secure credentials (AWS keys or environment variables) to provision resources. Application data is accessed through backend APIs with proper validation, and sensitive information is stored securely using environment configurations.

3.3 Data Retention Policies

- **Message Data Storage:** User messages and processed results are stored for a certain period to support analytics and monitoring of communication trends.
- **Deployment Logs:** CI/CD pipeline logs are maintained to track build and deployment activities for debugging and auditing purposes.
- **Infrastructure State:** Terraform state files are preserved to track infrastructure changes and support recovery in case of failures.
- **Security and Privacy:** Sensitive data is handled securely and can be removed or anonymized when no longer required.

3.4 Data Migration

Data and infrastructure migration in the system is managed using Infrastructure as Code (Terraform). By updating the provider configuration, the application and its resources can be moved from a local Docker environment to cloud platforms such as AWS EC2. Application data, including user messages and processed results, can also be migrated or backed up as needed to ensure continuity and system scalability.

4. Interfaces

• User Interface: Jenkins Web Dashboard for monitoring pipeline status.

• Command Line Interface (CLI): Terraform and Git CLI for developers.

5. State and Session Management

State management in the system is handled using the Terraform state file (.tfstate), which keeps track of all provisioned infrastructure resources. This ensures that Terraform can identify which resources need to be created, updated, or deleted during deployment.

Session management for the application is handled at the frontend and backend level, where user interactions and data are maintained during runtime to provide a smooth and consistent user experience.

6. Caching

- **Docker Layer Caching:** Docker uses layer caching to speed up the build process of application images, reducing deployment time in the CI/CD pipeline.
- **Dependency Caching:** The CI/CD pipeline caches dependencies and build files to avoid reinstallation during every build, improving efficiency.
- **Terraform Plugin Caching:** Terraform caches provider plugins locally, which helps in faster infrastructure provisioning and reduces execution time.

7. Functional Requirements

The functional requirements define the key features and services provided by the Non-Profit Support Triage System to ensure efficient message processing and automated deployment using DevOps practices. The system must support user interaction, message analysis, infrastructure automation, and monitoring through an integrated platform.

7.1 User Authentication & Authorization

- **Login System:** The system allows users or administrators to securely access the application dashboard.
- **Access Control:** Only authorized users can view or manage system data and deployment features.
- **Session Handling:** User sessions are maintained to ensure secure and continuous interaction with the system.

7.2 Application Configuration Management

- **Environment Configuration:** The system supports dynamic configuration using environment variables (e.g., API keys, cloud credentials).
- **Code Management:** Application code and configuration files are managed through GitHub for version control.

- **Data Storage:** Processed messages, results, and system data are stored for further analysis and monitoring.

7.3 Processing & Monitoring

- **Message Processing:** The system processes user messages by performing classification, urgency detection, and entity extraction.
- **Dashboard Monitoring:** A web-based dashboard displays processed results, message categories, and system insights.
- **Automated Processing:** The system automatically analyzes incoming data without manual intervention.

7.4 System Operations

- **Application Control:** The system allows starting, stopping, and managing application services through Docker containers.
- **Resource Management:** Infrastructure resources such as containers and cloud instances are managed efficiently.

7.5 Docker (Application Containerization)

- **Containerization:** The application is packaged using Docker to ensure consistent execution across development and production environments.
- **Portability:** Docker images allow easy deployment on local systems or cloud platforms.

7.6 CI/CD Pipeline (Workflow Configuration)

- **Automation:** CI/CD tools like GitHub Actions or Jenkins automatically build and deploy the application when code is pushed.
- **Image Creation:** Docker images are created and prepared for deployment during the pipeline process.

7.7 CI/CD Strategy (Continuous Integration & Deployment)

A. Continuous Integration (CI)

- **Automated Testing:** Every code push triggers a build process to check for syntax errors and ensure the application runs correctly.
- **Code Validation:** The pipeline verifies application code, Docker configuration, and IaC scripts to maintain quality.
- **Artifact Creation:** After successful testing, a Docker image is created as a reliable and consistent deployment unit.

B. Continuous Deployment (CD)

- **Infrastructure Provisioning:** Terraform updates and manages infrastructure on local Docker environments or AWS EC2 based on the latest configuration.

- **Container Deployment:** The Docker image is deployed and executed as containers in the target environment.
- **Reliable Updates:** The pipeline ensures smooth and continuous deployment with minimal downtime while updating the application.

8. Non-Functional Requirements

8.1 Security Aspects

- **Secure Configuration:** Application and infrastructure setup are automated to avoid manual misconfigurations.
- **Secret Management:** Sensitive data such as API keys and credentials are stored securely using environment variables or secret managers.
- **Access Control:** Only authorized users can access system features and deployment processes.

8.2 Performance Aspects

- **Efficiency:** Automation using CI/CD and Docker reduces deployment and setup time significantly.
- **Scalability:** The system can handle increasing user requests by deploying on cloud platforms like AWS.
- **Consistency:** Containerization ensures the same environment across development and production, eliminating configuration issues.

9. References

- **Problem Statement:** IaC Provisioning for Non-Profit Support System
- **Project Type:** DevOps-Based AI Application
- **Tools & Technologies:** Docker, Terraform, AWS, GitHub, CI/CD Pipelines

10. GitHub - https://github.com/NehaTamboli/carelink-project